

# Redis

---

Redis is like grease. It's most often used to lubricate moving parts and keep them working smoothly by reducing friction and speeding up their overall function. Whatever the machinery of your system, it could very well be improved with a bit poured over it. Sometimes the answer to your problem is simply a judicious use of more Redis.

First released in 2009, Redis (*RE*mote *D*ictionary *S*ervice) is a simple-to-use key-value store with a sophisticated set of commands. And when it comes to speed, Redis is hard to beat. Reads are fast and writes are even faster, handling upwards of 100,000 SET operations per second by some benchmarks. Redis creator Salvatore Sanfilippo (aka antirez) refers to his project as a “data structure server” to capture its nuanced handling of complex datatypes and other features. Exploring this super-fast, more-than-just-a-key-value-store will round out our view of the modern database landscape.

## Data Structure Server Store

It can be a bit difficult to classify exactly what Redis *is*. At a basic level, it's a key-value store, of course, but that simple label doesn't really do it justice. Redis supports advanced data structures, though not to the degree that a document-oriented database would. It supports set-based query operations but not with the granularity or type support you'd find in a relational database. And, of course, it's *fast*, trading durability for raw speed.

In addition to being an advanced data structure server, Redis can also be used as a blocking queue (or stack) and a publish-subscribe system. It features configurable expiry policies, durability levels, and replication options. All of this makes Redis more of a toolkit of useful data structure algorithms and processes than a member of any specific database genre.

Redis’s expansive list of client libraries makes it a drop-in option for many programming languages. It’s not simply easy to use; it’s a joy. If an API is UX for programmers, then Redis should be in the Museum of Modern Art alongside the Mac Cube.

In Days 1 and 2, we’ll explore Redis’s features, conventions, and configuration. Starting with simple CRUD operations, like always, we’ll quickly move on to more advanced operations involving more powerful data structures: lists, hashes, sets, and sorted sets. We’ll create transactions and manipulate data expiry characteristics. We’ll use Redis to create a simple message queue and explore its publish-subscribe functionality. Then we’ll dive into Redis’s configuration and replication options, learning how to strike an application-appropriate balance between data durability and speed.

Databases are often and increasingly used in concert with each other. Redis is introduced last in this book so that we can use it in just such a manner. In Day 3, we’ll build our capstone system, a rich multidatabase music solution including Redis, CouchDB, Neo4J, and Postgres—using Node.js to cement it together.

## Day 1: CRUD and Datatypes

Because the command-line interface (CLI) is of such primary importance to the Redis development team—and loved by users everywhere—we’re going to spend Day 1 looking at many of the 160+ commands available. Of primary importance are Redis’s sophisticated datatypes and how they can query in ways that go far beyond simply “retrieve the value of this key.”

### Getting Started

Redis is available through a few package builders such as Homebrew for Mac but is also rather painless to build from source.<sup>1</sup> We’ll be working off version 3.2.8. Once you have it installed, you can start up the server by calling this:

```
$ redis-server
```

It won’t run in the background by default, but you can make that happen by appending `&`, or you can just open another terminal. Next, run the command-line tool, which should connect to the default port 6379 automatically.

```
$ redis-cli
```

After you connect, let’s try to ping the server.

---

1. <http://redis.io>

```
redis 127.0.0.1:6379> PING
PONG
```

If you cannot connect, you'll receive an error message. Typing `help` will display a list of help options. Type `help` followed by a space and then start typing any command. If you don't know any Redis commands, just start pressing `Tab` to cycle through your options.

```
redis 127.0.0.1:6379> help
redis-cli 3.2.8
To get help about Redis commands type:
    "help @<group>" to get a list of commands in <group>
    "help <command>" for help on <command>
    "help <tab>" to get a list of possible help topics
    "quit" to exit
```

Today we're going to use Redis to build the back end for a URL shortener, such as [tinyurl.com](http://tinyurl.com) or [bit.ly](http://bit.ly). A URL shortener is a service that takes a really long URL and maps it to a shorter version on their own domain—like mapping <http://supercalifragilisticexpialidocious.com> to <http://bit.ly/VLD>. Visiting that short URL redirects users to the longer, mapped URL, which saves the visitors from text messaging long strings and also provides the short URL creator with some statistics, such as a count of visits.

In Redis, we can use `SET` to key a shortcode like `7wks` to a value like `http://www.sevenweeks.org`. `SET` always requires two parameters: a key and a value. Retrieving the value just needs `GET` and the key name.

```
redis 127.0.0.1:6379> SET 7wks http://www.sevenweeks.org/
OK
redis 127.0.0.1:6379> GET 7wks
"http://www.sevenweeks.org/"
```

To reduce traffic, we can also set multiple values with `MSET`, like any number of key-value pairs. Here we map `Google.com` to `gog` and `Yahoo.com` to `yah`.

```
redis 127.0.0.1:6379> MSET gog http://www.google.com yah http://www.yahoo.com
OK
```

Correlatively, `MGET` grabs multiple keys and returns values as an ordered list.

```
redis 127.0.0.1:6379> MGET gog yah
1) "http://www.google.com/"
2) "http://www.yahoo.com/"
```

Although Redis stores strings, it recognizes integers and provides some simple operations for them. If we want to keep a running total of how many short keys are in our dataset, we can create a count and then increment it with the `INCR` command.

```
redis 127.0.0.1:6379> SET count 2
OK
redis 127.0.0.1:6379> INCR count
(integer) 3
redis 127.0.0.1:6379> GET count
"3"
```

Although GET returns count as a string, INCR recognized it as an integer and added one to it. Any attempt to increment a non-integer ends poorly.

```
redis 127.0.0.1:6379> SET bad_count "a"
OK
redis 127.0.0.1:6379> INCR bad_count
(error) ERR value is not an integer or out of range
```

If the value can't be resolved to an integer, Redis rightly complains. You can also increment by any integer (INCRBY) or decrement (DECR, DECRBY).

## Transactions

You've seen transactions in previous databases (Postgres and Neo4j), and Redis's MULTI block atomic commands are a similar concept. Wrapping two operations like SET and INCR in a single block will complete either successfully or not at all. But you will never end up with a partial operation.

Let's key another shortcode to a URL and also increment the count all in one transaction. We begin the transaction with the MULTI command and execute it with EXEC.

```
redis 127.0.0.1:6379> MULTI
OK
redis 127.0.0.1:6379> SET prag http://pragprog.com
QUEUED
redis 127.0.0.1:6379> INCR count
QUEUED
redis 127.0.0.1:6379> EXEC
1) OK
2) (integer) 4
```

When using MULTI, the commands aren't actually executed when we define them (similar to Postgres transactions). Instead, they are queued and then executed in sequence.

Similar to ROLLBACK in SQL, you can stop a transaction with the DISCARD command, which will clear the transaction queue. Unlike ROLLBACK, it won't revert the database; it will simply not run the transaction at all. The effect is identical, although the underlying concept is a different mechanism (transaction rollback vs. operation cancellation).

## Complex Datatypes

So far, we haven't seen much complex behavior. Storing string and integer values under keys—even as transactions—is all fine and good, but most programming and data storage problems deal with many types of data. Storing lists, hashes, sets, and sorted sets natively helps explain Redis's popularity, and after exploring the complex operations you can enact on them, you may find you agree.

These collection datatypes can contain a huge number of values (up to  $2^{32}$  elements or more than 4 billion) per key. That's more than enough for all Facebook accounts to live as a list under a single key (though maybe not for much longer!).

While some Redis commands may appear cryptic, they generally follow a repeated pattern. SET commands begin with S, hashes with H, and sorted sets with Z. List commands generally start with either an L (for left) or an R (for right), depending on the direction of the operation (such as LPUSH).

### Hashes

Hashes are like nested Redis objects that can take any number of key-value pairs. Let's use a hash to keep track of users who sign up for our URL-shortening service.

Hashes are nice because they help you avoid storing data with artificial key prefixes. (Note that we will use colons [:] within our keys. This is a valid character that often logically separates a key into segments. It's merely a matter of convention, with no deeper meaning in Redis.)

```
redis 127.0.0.1:6379> MSET user:luc:name "Luc" user:luc:password s3cret
OK
redis 127.0.0.1:6379> MGET user:luc:name user:luc:password
1) "Luc"
2) "s3cret"
```

Instead of separate keys, we can create a hash that contains its own key-value pairs.

```
redis 127.0.0.1:6379> HMSET user:luc name "Luc" password s3cret
OK
```

We only need to keep track of the single Redis key to retrieve all values of the hash.

```
redis 127.0.0.1:6379> HVALS user:luc
1) "Luc"
2) "s3cret"
```

Or we can retrieve all hash keys.

```
redis 127.0.0.1:6379> HVALS user:luc
1) "name"
2) "password"
```

Or we can get a single value by passing in the Redis key followed by the hash key. Here we get just the password.

```
redis 127.0.0.1:6379> HGET user:luc password
"s3cret"
```

Unlike the document databases MongoDB and CouchDB, hashes in Redis cannot nest (nor can any other complex datatype such as lists). In other words, hashes can store only string values and not, say, sets or nested hashes.

More hash-specific commands exist to delete hash fields (HDEL), increment an integer field value by some count (HINCRBY), retrieve the number of fields in a hash (HLEN), get all keys *and* values (HGETALL), set a value only if the key doesn't yet exist (HSETNX), and more.

## Lists

Lists contain multiple ordered values that can act both as queues (first value in, first value out) and as stacks (last value in, first value out). They also have more sophisticated actions for inserting somewhere in the middle of a list, constraining list size, and moving values between lists.

Because our URL-shortening service can now track users, we want to allow them to keep a wishlist of URLs they'd like to visit. To create a list of short-coded websites we'd like to visit, we set the key to USERNAME:wishlist and push any number of values to the right (end) of the list.

```
redis 127.0.0.1:6379> RPUSh eric:wishlist 7wks gog prag
(integer) 3
```

Like most collection value insertions, the Redis command returns the number of values pushed. In other words, we pushed three values into the list so it returns 3. You can get the list length at any time with LLEN.

Using the list range command LRANGE, we can retrieve any part of the list by specifying the first and last positions. All list operations in Redis use a zero-based index. A negative position means the number of steps from the end.

```
redis 127.0.0.1:6379> LRANGE eric:wishlist 0 -1
1) "7wks"
2) "gog"
3) "prag"
```

LREM removes from the given key some matching values. It also requires a number to know how many matches to remove. Setting the count to 0 as we do here just removes them all:

```
redis 127.0.0.1:6379> LREM eric:wishlist 0 gog
```

Setting the count greater than 0 will remove only that number of matches, and setting the count to a negative number will remove that number of matches but scan the list from the end (right side).

To remove and retrieve each value in the order we added them (like a queue), we can pop them off from the left (head) of the list.

```
redis 127.0.0.1:6379> LPOP eric:wishlist
"7wks"
```

To act as a stack, after you R PUSH the values, you would RPOP from the end of the list. All of these operations are performed in constant time (meaning that the size of the list shouldn't impact performance).

On the previous combination of commands, you can use LPUSH and RPOP to similar effect (a queue) or LPUSH and LPOP to be a stack.

Suppose we wanted to remove values from our wishlist and move them to another list of visited sites. To execute this move atomically, we might try wrapping pop and push actions within a multiblock. In Ruby, these steps might look something like this (you can't use the CLI here because you must save the popped value, so we'll use the `redis-rb` gem):

```
redis.multi do
  site = redis.rpop('eric:wishlist')
  redis.lpush('eric:visited', site)
end
```

Because the multi block queues requests, the above won't work. This is really a feature of lists, not a bug because it prevents concurrent access to list members. Fortunately, Redis provides a single command for popping values from the tail of one list and pushing to the head of another. It's called RPOPLPUSH (right pop, left push).

```
redis 127.0.0.1:6379> RPOPLPUSH eric:wishlist eric:visited
"prag"
```

If you find the range of the wishlist, `prag` will be gone; it now lives under `visited`. This is a useful mechanism for queuing commands.

If you looked through the Redis docs to find RPOPRPUSH, LPOPLPUSH, and LPOPRPUSH commands, you may be dismayed to learn they don't exist. RPOPLPUSH is your only option, and you must build your list accordingly.

### Blocking Lists

Now that our URL shortener is taking off, let's add some social activities—like a real-time commenting system—where people can post about the websites they have visited.

Let's write a simple messaging system where multiple clients can push comments and one client (the digester) pops messages from the queue. We'd like the digester to just listen for new comments and pop them as they arrive. Redis provides a few blocking commands for this sort of purpose.

First, open another terminal and start another `redis-cli` client. This will be our digester. The command to block until a value exists to pop is `BRPOP`. It requires the key to pop a value from and a timeout in seconds, which we'll set to five minutes.

```
redis 127.0.0.1:6379> BRPOP comments 300
```

Then switch back to the first console and push a message to comments.

```
redis 127.0.0.1:6379> LPUSH comments "Prag is a great publisher!"
```

If you switch back to the digester console, two lines will be returned: the key and the popped value. The console will also output the length of time it spent blocking.

```
1) "comments"
2) "Prag is a great publisher!"
(7.88s)
```

There's also a blocking version of left pop (`BLPOP`) and left push (`BRPOPLPUSH`). `BLPOP` will remove the first element in a list or block (just like `BRPOP` removed the last element) while `BRPOPLPUSH` is a blocking version of `RPOPLPUSH`.

### Sets

Our URL shortener is shaping up nicely, but it would be nice to be able to group common URLs in some way.

Sets are unordered collections with no duplicate values and are an excellent choice for performing complex operations between two or more key values, such as unions or intersections.

If we wanted to categorize sets of URLs with a common key, we could add multiple values with `SADD`.



```
redis 127.0.0.1:6379> SADD news nytimes.com pragprog.com
(integer) 2
```

Redis added two values. We can retrieve the full set, in no particular order, using the SMEMBERS command.

```
redis 127.0.0.1:6379> SMEMBERS news
1) "pragprog.com"
2) "nytimes.com"
```

Let's add another category called *tech* for technology-related sites.

```
redis 127.0.0.1:6379> SADD tech pragprog.com apple.com
(integer) 2
```

To find the intersection of websites that both provide news and are technology focused, we use the SINTER command.

```
redis 127.0.0.1:6379> SINTER news tech
1) "pragprog.com"
```

Just as easily, we can remove any matching values in one set from another. To find all news sites that are not tech sites, use SDIFF:

```
redis 127.0.0.1:6379> SDIFF news tech
1) "nytimes.com"
```

We can also build a union of websites that are either news or tech. Because it's a set, any duplicates are dropped.

```
redis 127.0.0.1:6379> SUNION news tech
1) "apple.com"
2) "pragprog.com"
3) "nytimes.com"
```

That set of values can also be stored directly into a new set (SUNIONSTORE destination key [key ...]).

```
redis 127.0.0.1:6379> SUNIONSTORE websites news tech
redis 127.0.0.1:6379> SMEMBERS websites
1) "pragprog.com"
2) "nytimes.com"
3) "apple.com"
```

This also provides a useful trick for cloning a single key's values to another key, such as `SUNIONSTORE news_copy news`. Similar commands exist for storing intersections (SINTERSTORE) and diffs (SDIFFSTORE).

Just as RPOPLPUSH moved values from one list to another, SMOVE does the same for sets; it's just easier to remember. And like LLEN finds the length of a list, SCARD (set cardinality) counts the set; it's just harder to remember.

Because sets are not ordered, there are no left, right, or other positional commands. Popping a random value from a set just requires SPOP key, and removing values is SREM key value [value ...].

Unlike lists, there are no blocking commands for sets. With no blocking or positional commands, sets are a subpar choice for some Redis use cases such as message queues, but they're nonetheless great for all kinds of operations over collections.

### Sorted Sets

Whereas other Redis datatypes we've looked at so far easily map to common programming language constructs, sorted sets take something from each of the previous datatypes. They are ordered like lists and are unique like sets. They have field-value pairs like hashes, but their fields are numeric scores that denote the order of the values rather than plain strings. You can think of sorted sets as similar to a random access priority queue. This power has a trade-off, however. Internally, sorted sets keep values in order, so inserts can take  $\log(N)$  time to insert (where  $N$  is the size of the set), rather than the constant time complexity of hashes or lists.

Next, we want to keep track of the popularity of specific shortcodes. Every time someone visits a URL, the score gets increased. Like a hash, adding a value to a sorted set requires two values after the Redis key name: the score and the member.

```
redis 127.0.0.1:6379> ZADD visits 500 7wks 9 gog 9999 prag
(integer) 3
```

To increment a score, we can either re-add it with the new score, which just updates the score but does not add a new value, or increment by some number, which will return the new value.

```
redis 127.0.0.1:6379> ZINCRBY visits 1 prag
"10000"
```

You can decrement also by setting a negative number for ZINCRBY.

### Ranges

To get values from our visits set, we can issue a range command, ZRANGE, which returns by position, just like the list datatype's LRANGE command. Except in the case of a sorted set, the position is ordered by score from lowest to highest. So, to get the top two scoring visited sites (zero-based), use this:

```
redis 127.0.0.1:6379> ZRANGE visits 0 1
1) "gog"
2) "7wks"
```

To get the scores of each element as well, append `WITHSCORES` to the previous code. To get them in reverse, insert the word `REV`, as in `ZREVRANGE`.

```
redis 127.0.0.1:6379> ZREVRANGE visits 0 -1 WITHSCORES
1) "prag"
2) "10000"
3) "7wks"
4) "500"
5) "gog"
6) "9"
```

But if we're using a sorted set, it's more likely we want to range by score, rather than by position. `ZRANGEBYSCORE` has a slightly different syntax from `ZRANGE`. Because the low and high range numbers are *inclusive* by default, we can make a score number *exclusive* by prefixing it with an opening paren: (. So this will return all scores where  $9 \leq \text{score} \leq 10,000$ :

```
redis 127.0.0.1:6379> ZRANGEBYSCORE visits 9 9999
1) "gog"
2) "7wks"
```

But the following will return  $9 < \text{score} \leq 10,000$ :

```
redis 127.0.0.1:6379> ZRANGEBYSCORE visits (9 9999
1) "7wks"
```

We can also range by both positive and negative values, including infinities. This returns the entire set.

```
redis 127.0.0.1:6379> ZRANGEBYSCORE visits -inf inf
1) "gog"
2) "7wks"
3) "prag"
```

You can list them in reverse, too, with `ZREVRANGEBYSCORE`.

```
redis 127.0.0.1:6379> ZREVRANGEBYSCORE visits inf -inf
1) "prag"
2) "7wks"
3) "gog"
```

Along with retrieving a range of values by rank (index) or score, `ZREMRANGE-BYRANK` and `ZREMRANGEBYSCORE`, respectively, remove values by rank or score.

## Unions

Just like the set datatype, we can create a destination key that contains the union or intersection of one or more keys. This is one of the more complex commands in Redis because it must not only join the keys—a relatively simple operation—but also merge (possibly) differing scores. The union operation looks like this:

```
ZUNIONSTORE destination numkeys key [key ...]
  [WEIGHTS weight [weight ...]] [AGGREGATE SUM|MIN|MAX]
```

destination is the key to store into, and key is one or more keys to union. numkeys is simply the number of keys you're about to join, while weight is the optional number to multiply each score of the relative key by (if you have two keys, you can have two weights, and so on). Finally, aggregate is the optional rule for resolving each weighted score and summing by default, but you can also choose the min or max between many scores.

Let's use this command to measure the importance of a sorted set of shortcodes. First, we'll create another key that scores our shortcodes by votes. Each visitor to a site can vote if they like the site or not, and each vote adds a point.

```
redis 127.0.0.1:6379> ZADD votes 2 7wks 0 gog 9001 prag
(integer) 3
```

We want to figure out the most important websites in our system as some combination of votes and visits. Votes are important, but to a lesser extent, website visits also carry some weight (perhaps people are so enchanted by the website, they simply forget to vote). We want to add the two types of scores together to compute a new importance score while giving votes a weight of double importance—multiplied by two.

```
redis 127.0.0.1:6379> ZUNIONSTORE imp 2 visits votes WEIGHTS 1 2 AGGREGATE SUM
(integer) 3
redis 127.0.0.1:6379> ZRANGEBYSCORE imp -inf inf WITHSCORES
1) "gog"
2) "9"
3) "7wks"
4) "504"
5) "prag"
6) "28002"
```

This command is powerful in other ways, too. For example, if you need to double all scores of a set, you can union a single key with a weight of 2 and store it back into itself.

```
redis 127.0.0.1:6379> ZUNIONSTORE votes 1 votes WEIGHTS 2
(integer) 2
redis 127.0.0.1:6379> ZRANGE votes 0 -1 WITHSCORES
1) "gog"
2) "0"
3) "7wks"
4) "4"
5) "prag"
6) "18002"
```

Sorted sets contain a similar command (ZINTERSTORE) to perform intersections.

## Expiry

Another common use case for a key-value system like Redis is as a fast-access cache for data that's more expensive to retrieve or compute. In just about any cache, ensuring that keys expire after a designated time period is essential to keeping the key set from growing unboundedly.

Marking a key for expiration requires the EXPIRE command, an existing key, and a time to live (in seconds). Here we set a key and set it to expire in ten seconds. We can check whether the key EXISTS within ten seconds and it returns a 1 (true). If we wait to execute, it will eventually return a 0 (false).

```
redis 127.0.0.1:6379> SET ice "I'm melting..."
OK
redis 127.0.0.1:6379> EXPIRE ice 10
(integer) 1
redis 127.0.0.1:6379> EXISTS ice
(integer) 1
redis 127.0.0.1:6379> EXISTS ice
(integer) 0
```

Setting and expiring keys is so common that Redis provides a shortcut command called SETEX.

```
redis 127.0.0.1:6379> SETEX ice 10 "I'm melting..."
```

You can query the time a key has to live with TTL. Setting ice to expire as shown earlier and checking its TTL will return the number of seconds left (or -2 if the key has already expired or doesn't exist, which is the same thing).

```
redis 127.0.0.1:6379> TTL ice
(integer) 4
```

At any moment before the key expires, you can remove the timeout by running PERSIST key.

```
redis 127.0.0.1:6379> PERSIST ice
```

For marking a countdown to a specific time, `EXPIREAT` accepts a Unix timestamp (as seconds since January 1, 1970) rather than a number of seconds to count up to. In other words, `EXPIREAT` is for absolute timeouts, and `EXPIRE` is for relative timeouts.

A common trick for keeping only recently used keys is to update the expire time whenever you retrieve a value. This is the most recently used (MRU) caching algorithm to ensure that your most recently used keys will remain in Redis, while the unused keys will just expire as normal.

## Database Namespaces

So far, we've interacted only with a single namespace. Sometimes we need to separate keys into multiple namespaces. For example, if you wrote an internationalized key-value store, you could store different translated responses in different namespaces. The key `greeting` could be set to `"guten Tag"` in a German namespace and `"bonjour"` in French. When a user selects their language, the application just pulls all values from the namespace assigned.

In Redis nomenclature, a namespace is called a *database* and is keyed by number. So far, we've always interacted with the default namespace 0 (also known as database 0). Here we set `greeting` to the English `hello`.

```
redis 127.0.0.1:6379> SET greeting hello
OK
redis 127.0.0.1:6379> GET greeting
"hello"
```

But if we switch to another database via the `SELECT` command, that key is unavailable.

```
redis 127.0.0.1:6379> SELECT 1
OK
redis 127.0.0.1:6379[1]> GET greeting
(nil)
```

And setting a value to this database's namespace will not affect the value of the original.

```
redis 127.0.0.1:6379[1]> SET greeting "guten Tag"
OK
redis 127.0.0.1:6379[1]> SELECT 0
OK
redis 127.0.0.1:6379> GET greeting
"hello"
```

Because all databases are running in the same server instance, Redis lets us shuffle keys around with the `MOVE` command. Here we move `greeting` to database 2:

```
redis 127.0.0.1:6379> MOVE greeting 2
(integer) 2
redis 127.0.0.1:6379> SELECT 2
OK
redis 127.0.0.1:6379[2]> GET greeting
"hello"
```

This can be useful for running different applications against a single Redis server while still allowing these multiple applications to trade data between each other.

## And There's More

Redis has plenty of other commands for actions such as renaming keys (`RENAME`), determining the type of a key's value (`TYPE`), and deleting a key-value (`DEL`). There's also the painfully dangerous `FLUSHDB`, which removes all keys from this Redis database, and its apocalyptic cousin, `FLUSHALL`, which removes all keys from all Redis databases. We won't cover all commands here, but we recommend checking out the online documentation for the full list of Redis commands (and we must say, the Redis documentation is truly a joy to read and interact with).

## Day 1 Wrap-Up

The datatypes of Redis and the complex queries it can perform make it much more than a standard key-value store. It can act as a stack, queue, or priority queue; you can interact with it as you would an object store (via hashes); and Redis even can perform complex set operations such as unions, intersections, and subtractions (`diff`). It provides many atomic commands as well as a transaction mechanism for multistep commands. Plus, it has a built-in ability to expire keys, which is useful as a cache.

## Day 1 Homework

### Find

Find the complete Redis commands documentation, as well as the Big-O notated ( $O(x)$ ) time complexity under the command details.

## Do

1. Install your favorite programming language driver and connect to the Redis server. Insert and increment a value within a transaction.
2. Using your driver of choice, create a program that reads a blocking list and outputs somewhere (console, file, Socket.io, and so on) and another that writes to the same list.



## Day 2: Advanced Usage, Distribution

Day 1 introduced us to Redis as a data structure server. Today, we'll build on that foundation by looking at some of the advanced functions provided by Redis, such as pipelining, the publish-subscribe model, system configuration, and replication. Beyond that, we'll look at how to create a Redis cluster, store a lot of data quickly, and use an advanced technique involving Bloom filters.

### A Simple Interface

At 80,000 lines of source code, Redis is a fairly small project compared to most databases. But beyond code size, it has a simple interface that accepts the very strings we have been writing in the console.

#### Telnet

We can interact without the command-line interface by streaming commands through TCP on our own via telnet and terminating the command with a carriage return line feed (CRLF, or `\r\n`). Run `Ctrl+J` at any time to exit.

```
redis/telnet.sh
$ telnet localhost 6379
Trying 127.0.0.1...
Connected to localhost.
Escape character is '^J'.
SET test hello
+OK
GET test
```



```
$5
hello
SADD stest 1 99
:2
SMEMBERS stest
*2
$1
1
$2
99
```

Here we see four Redis commands as inputs (you should be able to identify them quickly) and their corresponding outputs. We can see that our input is the same as we provided in the Redis console, but the console has cleaned up the responses a bit. To give a few examples:

- Redis streams the OK status prefixed by a + sign.
- Before it returned the string *hello*, it sent \$5, which means “the following string is five characters.”
- After we add two set items to the test key, the number 2 is prefixed by : to represent an integer (two values were added successfully).

Finally, when we requested two items, the first line returned begins with an asterisk and the number 2—meaning there are two complex values about to be returned. The next two lines are just like the *hello* string but contain the string *1*, followed by the string *99*.

## Pipelining

We can also stream our own strings one at a time by using the BSD netcat (nc) command, which is already installed on many Unix machines. With netcat, we must specifically end a line with CRLF (telnet did this for us implicitly). We also sleep for a second after the ECHO command has finished to give some time for the Redis server to return. Some nc implementations have a -q option, thus negating the need for a sleep, but not all do, so feel free to try it.

```
$ (echo -en "ECHO hello\r\n"; sleep 1) | nc localhost 6379
$5
hello
```

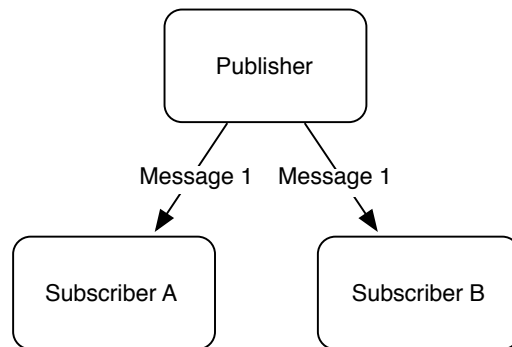
We can take advantage of this level of control by *pipelining* our commands, or streaming multiple commands in a single request.

```
$ (echo -en "PING\r\nPING\r\nPING\r\n"; sleep 1) | nc localhost 6379
+PONG
+PONG
+PONG
```

This can be far more efficient than pushing a single command at a time and should always be considered if it makes sense to do so—especially in transactions. Just be sure to end every command with `\r\n`, which is a required delimiter for the server.

## publish-subscribe

On Day 1, we were able to implement a rudimentary blocking queue using the list datatype. We queued data that could be read by a blocking pop command. Using that queue, we made a very basic queue according to a publish-subscribe model. Any number of messages could be pushed to this queue, and a single queue reader would pop messages as they were available. This is powerful but limited. Under many circumstances we want a slightly inverted behavior, where several subscribers want to read the announcements of a single publisher who sends a message to all subscribers, as shown in the following figure. Redis provides some specialized publish-subscribe (or pub-sub) commands.



Let's improve on the commenting mechanism we made before using blocking lists, by allowing a user to post a comment to multiple subscribers (as opposed to just one). We start with a subscriber that listens on a key for messages (the key will act as a *channel* in pub-sub nomenclature). This will cause the CLI to output Reading messages... and then block while the subscriber listens for incoming messages.

```

redis 127.0.0.1:6379> SUBSCRIBE comments
Reading messages... (press Ctrl-C to quit)
1) "subscribe"
2) "comments"
3) (integer) 1
  
```

With two subscribers, we can publish any string we want as a message to the comments channel. The PUBLISH command will return the integer 2, meaning two subscribers received it.

```
redis 127.0.0.1:6379> PUBLISH comments "Check out this shortcoded site! 7wks"
(integer) 2
```

Both of the subscribers will receive a *multibulk reply* (a list) of three items: the string “message,” the channel name, and the published message value.

- 1) "message"
- 2) "comments"
- 3) "Check out this shortcoded site! 7wks"

When your clients no longer want to receive correspondence, they can execute the `UNSUBSCRIBE comments` command to disconnect from the comments channel or simply `UNSUBSCRIBE` alone to disconnect from all channels. However, note that in the `redis-cli` console you will have to press `Ctrl+C` to break the connection.

## Server Info

Before getting into changing Redis’s system settings, it’s worth taking a quick look at the `INFO` command because changing settings values will alter some of these values as well. `INFO` outputs a list of server data, including version, process ID, memory used, and uptime.

```
redis 127.0.0.1:6379> INFO
# Server
redis_version:3.2.8
redis_git_sha1:00000000
redis_git_dirty:0
redis_build_id:b533f811ec736a0c
redis_mode:standalone
...
```

You may want to revisit this command again in this chapter because it provides a useful snapshot of this server’s global information and settings. It even provides information on durability, memory fragmentation, and replication server status.

## Redis Configuration

So far, we’ve only used Redis with its out-of-the-box configuration. But much of Redis’s power comes from its configurability, allowing you to tailor settings to your use case. The `redis.conf` file that comes with the distribution—found in `/etc/redis` on \*nix systems or `/usr/local/etc` on Mac OS—is fairly self-explanatory, so we’re going to cover only a portion of the file. We’ll go through a few of the common settings in order.

```
daemonize no
port 6379
loglevel verbose
logfile stdout
database 16
```

By default, `daemonize` is set to *no*, which is why the server always starts up in the foreground. This is nice for testing but not very production friendly. Changing this value to *yes* will run the server in the background while setting the server's process ID in a pid file.

The next line is the default port number for this server, port 6379. This can be especially useful when running multiple Redis servers on a single machine.

`loglevel` defaults to *verbose*, but it's good to set it to *notice* or *warning* in production to cut down on the number of log events. `logfile` outputs to *stdout* (standard output, the console), but a filename is necessary if you run in *daemonize* mode.

`database` sets the number of Redis databases we have available. We saw how to switch between databases. If you plan to only ever use a single database namespace, it's not a bad idea to set this to 1 to prevent unwanted databases from being accidentally created.

## Durability

Redis has a few persistence options. First is no persistence at all, which will simply keep all values in main memory. If you're running a basic caching server, this is a reasonable choice since durability always increases latency.

One of the things that sets Redis apart from other fast-access caches like `memcached`<sup>2</sup> is its built-in support for storing values to disk. By default, key-value pairs are only occasionally saved. You can run the `LASTSAVE` command to get a Unix timestamp of the last time a Redis disk write succeeded, or you can read the `last_save_time` field from the server `INFO` output.

You can force durability by executing the `SAVE` command (or `BGSAVE`, to asynchronously save in the background).

```
redis 127.0.0.1:6379> SAVE
```

If you read the `redis-server` log, you will see lines similar to this:

```
[46421] 10 Oct 19:11:50 * Background saving started by pid 52123
[52123] 10 Oct 19:11:50 * DB saved on disk
[46421] 10 Oct 19:11:50 * Background saving terminated with success
```

---

2. <http://www.memcached.org/>

Another durability method is to alter the snapshotting settings in the configuration file.

### Snapshotting

We can alter the rate of storage to disk by adding, removing, or altering one of the save fields. By default, there are three, prefixed by the save keyword followed by a time in seconds and a minimum number of keys that must change before a write to disk occurs. For example, to trigger a save every 5 minutes (300 seconds) if any keys change at all, you would write the following:

```
save 300 1
```

The configuration has a good set of defaults. The set means if 10,000 keys change, save in 60 seconds; if 10 keys change, save in 300 seconds, and any key changes will be saved in at least 900 seconds (15 minutes).

```
save 900 1
save 300 10
save 60 10000
```

You can add as many or as few save lines as necessary to specify precise thresholds.

### Append-Only File

Redis is *eventually durable* by default, in that it asynchronously writes values to disk in intervals defined by our save settings, or it is forced to write by client-initiated commands. This is acceptable for a second-level cache or session server but is insufficient for storing data that you need to be durable, like financial data. If a Redis server crashes, our users might not appreciate having lost money.

Redis provides an append-only file (appendonly.aof) that keeps a record of all write commands. This is like the write-ahead logging we saw in [Chapter 3, HBase, on page 53](#). If the server crashes before a value is saved, it executes the commands on startup, restoring its state; appendonly must be enabled by setting it to yes in the redis.conf file.

```
appendonly yes
```

Then we must decide how often a command is appended to the file. Setting always is the more durable because every command is saved. It's also slow, which often negates the reason people have for using Redis. By default, everysec is enabled, which saves up and writes commands only once a second. This is a decent trade-off because it's fast enough, and worst case you'll lose only the last one second of data. Finally, no is an option, which just lets the

OS handle flushing. It can be fairly infrequent, and you're often better off skipping the append-only file altogether rather than choosing it.

```
# appendfsync always
appendfsync everysec
# appendfsync no
```

Append-only has more detailed parameters, which may be worth reading about in the config file when you need to respond to specific production issues.

## Security

Although Redis is not natively built to be a fully secure server, you may run across the `requirepass` setting and `AUTH` command in the Redis documentation. These can be safely ignored because they are merely a scheme for setting a plaintext password. Because a client can try nearly 100,000 passwords a second, it's almost a moot point, beyond the fact that plaintext passwords are inherently unsafe anyway. If you want Redis security, you're better off with a good firewall and SSH security.

Interestingly, Redis provides command-level security through obscurity, by allowing you to hide or suppress commands. Adding this to your config will rename the `FLUSHALL` command (remove all keys from the system) into some hard-to-guess value like `c283d93ac9528f986023793b411e4ba2`:

```
rename-command FLUSHALL c283d93ac9528f986023793b411e4ba2
```

If you attempt to execute `FLUSHALL` against this server, you'll be hit with an error. The secret command works instead.

```
redis 127.0.0.1:6379> FLUSHALL
(error) ERR unknown command 'FLUSHALL'
redis 127.0.0.1:6379> c283d93ac9528f986023793b411e4ba2
OK
```

Or better yet, we can disable the command entirely by setting it to a blank string in the configuration.

```
rename-command FLUSHALL ""
```

You can set any number of commands to a blank string, allowing you a modicum of customization over your command environment.

## Tweaking Parameters

There are several more advanced settings for speeding up slow query logs, encoding details, making latency tweaks, and importing external config files. Keep in mind, though, if you run across some documentation about Redis virtual memory, simply ignore it, as that feature was removed in version 2.6.

To aid in testing your server configuration, Redis provides an excellent benchmarking tool. It connects locally to port 6379 by default and issues 10,000 requests using 50 parallel clients. We can execute 100,000 requests with the `-n` argument.

```
$ redis-benchmark -n 100000
===== PING (inline) =====
  100000 requests completed in 0.89 seconds
   50 parallel clients
   3 bytes payload
  keep alive: 1
99.91% <= 1 milliseconds
100.00% <= 1 milliseconds
32808.40 requests per second
...
```

Other commands are tested as well, such as `SADD` and `LRANGE`, with the more complex commands generally taking more time.

## Master-Slave Replication

As with other NoSQL databases we've seen (such as MongoDB and Neo4j), Redis supports master-slave replication. One server is the master by default if you don't set it as a slave of anything. Data will be replicated to any number of slave servers.

Making slave servers is easy. We first need a copy of our `redis.conf` file.

```
$ cp redis.conf redis-s1.conf
```

The file will remain largely the same but with the following changes:

```
port 6380
slaveof 127.0.0.1 6379
```

If all went according to plan, you should see something similar to the following in the slave server's log when you start it:

```
$ redis-server redis-s1.conf

34648:S 28 Apr 06:42:22.496 * Connecting to MASTER 127.0.0.1:6379
34648:S 28 Apr 06:42:22.496 * MASTER <-> SLAVE sync started
34648:S 28 Apr 06:42:22.497 * Non blocking connect for SYNC fired the event.
34648:S 28 Apr 06:42:22.497 * Master replied to PING, replication can...
34648:S 28 Apr 06:42:22.497 * Partial resynchronization not possible...
34648:S 28 Apr 06:42:22.497 * Full resync from master: 4829...1f88a68bc:1
34648:S 28 Apr 06:42:22.549 * MASTER <-> SLAVE sync: receiving 76 bytes...
34648:S 28 Apr 06:42:22.549 * MASTER <-> SLAVE sync: Flushing old data
34648:S 28 Apr 06:42:22.549 * MASTER <-> SLAVE sync: Loading DB in memory
34648:S 28 Apr 06:42:22.549 * MASTER <-> SLAVE sync: Finished with success
```

And you should see the string 1 slaves output in the master log.

```
redis 127.0.0.1:6379> SADD meetings "StarTrek Pastry Chefs" "LARPers Intl."
```

If we connect the command line to our slave, we should receive our meeting list.

```
redis 127.0.0.1:6380> SMEMBERS meetings
1) "StarTrek Pastry Chefs"
2) "LARPers Intl."
```

In production, you'll generally want to implement replication for availability or backup purposes and thus have Redis slaves on different machines.

## Data Dump

So far, we've talked a lot about how fast Redis is, but it's hard to get a feel for it without playing with a bit more data.

Let's insert a large dummy dataset into our Redis server. You can keep the slave running if you like, but a laptop or desktop might run quicker if you're running just a single master server. We're going to autogenerate a list of keys and values of arbitrary size, where the keys will be key1, key2, and so on, while the values will be value1, and so on.

You'll first need to install the redis Ruby gem.

```
$ gem install redis
```

There are several ways to go about inserting a large dataset, and they get progressively faster but more complex.

The simplest method is to simply iterate through a list of data and execute SET for each value using the standard redis-rb client. In our case, we don't really care what the data looks like, as we just want to look at performance, so we'll insert our randomized data.

```
redis/data_dump.rb
require 'redis'
#%w{hiredis redis/connection/hiredis}.each{|r| require r}
# the number of set operations to perform will be defined as a CLI arg
TOTAL_NUMBER_OF_ENTRIES = ARGV[0].to_i
$redis = Redis.new(:host => "127.0.0.1", :port => 6379)
$redis.flushall
count, start = 0, Time.now
(1..TOTAL_NUMBER_OF_ENTRIES).each do |n|
  count += 1
```



```

key = "key#{n}"
value = "value#{n}"

$redis.set(key, value)

# stop iterating when we reach the specified number
break if count >= TOTAL_NUMBER_OF_ENTRIES
end
puts "#{count} items in #{Time.now - start} seconds"

```

Run the script, specifying the number of SET operations. Feel free to experiment with lower or higher numbers. Let's start with 100,000.

```

$ ruby data_dump.rb 100000
100000 items in 5.851211 seconds

```

If you want to speed up insertion—and are not running JRuby—you can optionally install the hiredis gem. It's a C driver that is considerably faster than the native Ruby driver. Just uncomment the `%w{hiredis redis/connection/hiredis}.each{|r| require r}` statement at the top in order to load the driver and then re-run the script. You may not see a large improvement for this type of CPU-bound operation, but we highly recommend hiredis for production Ruby use.

You will, however, see a big improvement with pipelined operations. Here we batch 1,000 lines at a time and pipeline their insertion. You may see time reductions of 500 percent or more.

```

redis/data_dump_pipelined.rb
require 'redis'
#%w{hiredis redis/connection/hiredis}.each{|r| require r}

TOTAL_NUMBER_OF_ENTRIES = ARGV[0].to_i
BATCH_SIZE = 1000

# perform a single batch update for each number
def flush(batch)
  $redis.pipelined do
    batch.each do |n|
      key, value = "key#{n}", "value#{n}"
      $redis.set(key, value)
    end
  end
  batch.clear
end

$redis = Redis.new(:host => "127.0.0.1", :port => 6379)
$redis.flushall

batch = []
count, start = 0, Time.now

```

```

(1..TOTAL_NUMBER_OF_ENTRIES).each do |n|
  count += 1

  # push integers into an array
  batch << n

  # watch this number fluctuate between 1 and 1000
  puts "Batch size: #{batch.length}"

  # if the array grows to BATCH_SIZE, flush it
  if batch.size == BATCH_SIZE
    flush(batch)
  end

  break if count >= TOTAL_NUMBER_OF_ENTRIES
end
# flush any remaining values
flush(batch)

puts "#{count} items in #{Time.now - start} seconds"

$ ruby data_dump_pipelined.rb 100000
100000 items in 1.061089 seconds

```

This reduces the number of Redis connections required, but building the pipelined dataset has some overhead of its own. You should try it out with different numbers of batched operations when pipelining in production. For now, experiment with increasing the number of items and re-run the script using the hiredis gem for an even more dramatic performance increase.

## Redis Cluster

Beyond simple replication, many Redis clients provide an interface for building a simple ad hoc distributed Redis cluster. The Ruby client supports a consistent-hashing managed cluster.

To get started with building out a managed cluster, we need another server. Unlike the master-slave setup, both of our servers will take the master (default) configuration. We copied the redis.conf file and changed the port to 6380. That's all that's required for the servers.

```

redis/data_dump_cluster.rb
require 'redis'
require 'redis/distributed'

TOTAL_NUMBER_OF_ENTRIES = ARGV[0].to_i

$redis = Redis::Distributed.new([
  "redis://localhost:6379/",
  "redis://localhost:6380/"
])
$redis.flushall
count, start = 0, Time.now

```

```
(1..TOTAL_NUMBER_OF_ENTRIES).each do |n|
  count += 1

  key = "key#{n}"
  value = "value#{n}"

  $redis.set(key, value)

  break if count >= TOTAL_NUMBER_OF_ENTRIES
end
puts "#{count} items in #{Time.now - start} seconds"
```

Bridging between two or more servers requires only some minor changes to our existing data dump client. First, we need to require the `redis/distributed` file from the `redis` gem.

```
require 'redis/distributed'
```

Then replace the `Redis` client with `Redis::Distributed` and pass in an array of server URIs. Each URI requires the `redis` scheme, server (localhost in our case), and port.

```
$redis = Redis::Distributed.new([
  "redis://localhost:6379/",
  "redis://localhost:6380/"
])
```

Running the client is the same as before.

```
$ ruby data_dump_cluster.rb 10000
100000 items in 6.614907 seconds
```

We do see a performance decrease here because a lot more work is being done by the client, since it handles computing which keys are stored on which servers. You can validate that keys are stored on separate servers by attempting to retrieve the same key from each server through the CLI.

```
$ redis-cli -p 6379 --raw GET key537
$ redis-cli -p 6380 --raw GET key537
```

Only one client will be able to GET the value of `value537`. But as long as you retrieve keys set through the same `Redis::Distributed` configuration, the client will access the values from the correct servers.

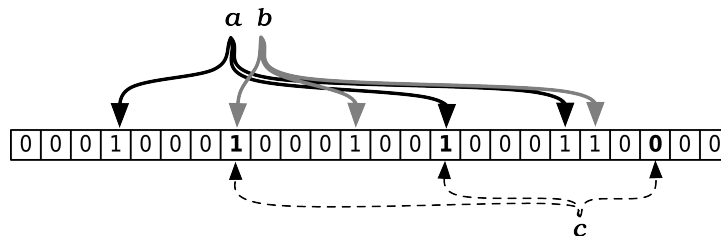
## Bloom Filters

A good way to improve the performance of just about *any* data retrieval system is to simply never perform queries that you *know* are doomed to fail and find no data. If you know that you dropped your car keys in your house, it's senseless to scour your neighbor's house in search of them. You should start

your search operation at home, maybe starting with the couch cushions. What matters is that you can safely exclude some search avenues that you know will be fruitless.

A *Bloom filter* enables you to do something similar with database queries. These filters are probabilistic data structures that check for the nonexistence of an item in a set, first covered in [Compression and Bloom Filters, on page 71](#). Although Bloom filters can return false positives, they cannot return a false negative. This is very useful when you need to quickly discover whether a value does not exist in a system. If only they made Bloom filters for lost car keys!

Bloom filters succeed at discovering nonexistence by converting a value to a very sparse sequence of bits and comparing that to a union of every value's bits. In other words, when a new value is added, it is OR'd against the current Bloom filter bit sequence. When you want to check whether the value is already in the system, you perform an AND against the Bloom filter's sequence. If the value has any true bits that aren't also true in the Bloom filter's corresponding buckets, then the value was never added. In other words, this value is definitely not in the Bloom filter. The following figure provides a graphic representation of this concept.



To get started writing our own Bloom filter, we need to install a new gem:

```
$ gem install bloomfilter-rb
```

Ruby wunderkind Ilya Grigorik created this Redis-backed Bloom filter, but the concepts are transferable to any language. Let's have a look at a script that looks a bit like our previous data dump script but with a few key differences.

For our example here, we'll download a text file containing the entire text of *Moby Dick* from Project Gutenberg<sup>3</sup> and assemble a list of all words in the text (including a *lot* of repeat words such as “the” and “a”). Then, we'll loop through each word, check if it's already in our Bloom filter, and insert it into the filter if it isn't there already.

3. <https://www.gutenberg.org/>

```

redis/bloom_filter.rb
require 'bloomfilter-rb'

bloomfilter = BloomFilter::Redis.new(:size => 1000000)
bloomfilter.clear

# we'll read the file data and strip out all the non-word material
text_data = File.read(ARGV[0])
clean_text = text_data.gsub(/\n/, ' ').gsub(/[.,-;'?"()!*]/, '')

clean_text.split(' ').each do |word|
  word = word.downcase

  next if bloomfilter.include?(word)
  puts word
  bloomfilter.insert(word)
end

puts "Total number of words: #{text_data.length}"
puts "Number of words in filter: #{bloomfilter.size}"

```

Let's download the text using cURL and run the script:

```

$ curl -o moby-dick.txt https://www.gutenberg.org/files/2701/old/moby10b.txt
$ ruby bloom_filter.rb moby-dick.txt > output.txt

```

Open up the output.txt and scroll through the contents. Each word in this file has not yet been added to the filter. At the top of the list, you'll find a lot of common words like *the*, *a*, and *but*. At the bottom of the list, you'll see the word "orphan," which is the very last word in the Epilogue, which explains why it hadn't been added to the filter yet! Some other fairly esoteric words toward the end include "ixion," "sheathed," "dirgelike," and "intermixingly."

What essentially happened here is that the more frequently used words were more likely to get filtered out early, whereas less common words or words used only once were filtered out later. The upside with this approach is the ability to detect duplicate words. The downside is that a few false positives will seep through—the Bloom filter may flag a word we have never seen before. This is why in a real-world use case you would perform some secondary check, such as a slower database query to a system of record, which should happen only a small percentage of the time, presuming a large enough filter size, which is computable.<sup>4</sup>

## SETBIT and GETBIT

As mentioned earlier, Bloom filters function by flipping certain bits in a sparse binary field. The Redis Bloom filter implementation we just used takes advantage of two Redis commands that perform just such actions: SETBIT and GETBIT.

4. [http://en.wikipedia.org/wiki/Bloom\\_filter](http://en.wikipedia.org/wiki/Bloom_filter)

Like all Redis commands, SETBIT is fairly descriptive. The command sets a single bit (either 1 or 0) at a certain location in a bit sequence, starting from zero. It's a common use case for high-performance multivariate flagging—it's faster to flip a few bits than write a set of descriptive strings.

If we want to keep track of the toppings on a hamburger, we can assign each type of topping to a bit position, such as ketchup = 0, mustard = 1, onion = 2, lettuce = 3. So, a hamburger with only mustard and onion could be represented as 0110 and set in the command line:

```
redis 127.0.0.1:6379> SETBIT my_burger 1 1
(integer) 0
redis 127.0.0.1:6379> SETBIT my_burger 2 1
(integer) 0
```

Later, a process can check whether my burger should have lettuce or mustard. If zero is returned, the answer is false—one if true.

```
redis 127.0.0.1:6379> GETBIT my_burger 3
(integer) 0
redis 127.0.0.1:6379> GETBIT my_burger 1
(integer) 1
```

The Bloom filter implementation takes advantage of this behavior by hashing a value as a multibit value. It calls SETBIT X 1 for each on position in an insert() (where X is the bit position) and verifies existence by calling GETBIT X on the include?() method—returning false if any GETBIT position returns 0.

Bloom filters are excellent for reducing unnecessary traffic to a slower underlying system, be it a slower database, limited resource, or network request. If you have a slower database of IP addresses and you want to track all new users to your site, you can use a Bloom filter to first check whether the IP address exists in your system. If the Bloom filter returns false, you know the IP address has yet to be added and can respond accordingly. If the Bloom filter returns true, this IP address may or may not exist on the back end and requires a secondary lookup to be sure. This is why computing the correct size is important—a well-sized Bloom filter can reduce (but not eliminate) the error rate or the likelihood of a false positive.

## Day 2 Wrap-Up

Today we rounded out our Redis investigation by moving beyond simple operations into squeezing every last bit of speed out of an already very fast system. Redis provides for fast and flexible data structure storage and simple manipulations as we saw in Day 1, but it's equally adept at more complex behaviors by way of built-in publish-subscribe functions and bit operations.

It's also highly configurable, with many durability and replication settings that conform to whatever your needs may be. Finally, Redis also supports some nice third-party enhancements, such as Bloom filters and clustering.

This concludes major operations for the Redis data structure store. Tomorrow we're going to do something a bit different, by using Redis as the cornerstone of a polyglot persistence setup along with CouchDB and Neo4j.

## Day 2 Homework

### Find

1. Find out what messaging patterns are, and discover how many Redis can implement.
2. Read some documentation on Sentinel,<sup>5</sup> a system used to manage high-availability Redis clusters.

### Do

1. Run the data dump script with all snapshotting and the append-only file turned off. Then try running with `appendfsync` set to `always`, noting the speed difference.
2. Using your favorite programming language's web framework, try to build a simple URL-shortening service backed by Redis with an input box for the URL and a simple redirect based on the URL. Back it up with a Redis master-slave replicated cluster across multiple nodes as your back end.

## Day 3: Playing with Other Databases

Today we're wrapping up our final database chapter by inviting some other databases from the book to the Redis party. Amongst the databases we've gone over in this book, Redis is the most lightweight and modular, capable of enhancing just about any complex data setup without imposing a ton of engineering and ops overhead. Redis will hold a starring role in today's exercise by making our interaction with other databases faster and easier.

We've learned throughout this book that different databases have different strengths, so many modern systems have moved toward a *polyglot persistence model*, where many databases each play a role in the system. You will learn how to build one of these projects using CouchDB as the system of record (the canonical data source), Neo4j to handle data relationships, and Redis

---

5. <http://objectrocket.com/blog/how-to/introduction-to-redis-sentinel>

to help with data population and caching. Consider this your final exam for the book.

Note that our choice of databases for this last project should not be seen as the authors' endorsement of any specific set of databases, languages, or frameworks over another but rather a showcase of how multiple databases can work together, leveraging the capabilities of each in pursuit of a single goal.

## A Polyglot, Persistent Service

Our polyglot persistence service will act as the backend for a web interface that provides information about bands. We want to store a list of musical band names, the artists who performed in those bands, and any number of roles each artist played in the band, from lead singer to backup keytar player to people who just dance (like the guy in Mighty Mighty Bosstones). Each of the three databases—Redis, CouchDB, and Neo4j—will handle a different aspect of our band data management system.

Redis plays three important roles in our system: It assists in populating data to CouchDB, it acts as a cache for recent Neo4j changes, and it enables quick lookup for partial value searches. Its speed and ability to store multiple data formats make it well suited for data population, and its built-in expiry policies are perfect for handling cached data.

CouchDB is our system of record (SOR), or authoritative data source. CouchDB's document structure is an easy way to store band data because it allows for nested artist and role information, and we will take advantage of the changes API<sup>6</sup> in CouchDB to keep our third data source in sync.

Neo4j is our relationship store. Although querying the CouchDB SOR directly is perfectly reasonable, a graph database affords us a simplicity and speed in walking node relationships that other databases have a difficult time matching. We'll store relationships between bands, band members, and the roles the members play.

Each database has a specific role to play in our system, but they don't natively communicate with another, which means that we need to build a translation layer between them. We'll use Node.js to populate the databases, communicate between them, and act as a simple front-end server. Because gluing multiple databases together requires a bit of code, this last day will include much more code than we have seen so far in this book.

---

6. <http://docs.couchdb.org/en/latest/api/database/changes.html>



## The Rise of Polyglot Persistence

Like the growing phenomenon of polyglot programming, polyglot persistence is now well established in most corners of the database universe.

If you are unfamiliar with the practice, polyglot programming is what happens when a team uses more than one programming language in a single project. Contrast this with the convention of using one general-purpose language throughout a project or even an entire company (you may have heard of “Java shops” or “Ruby shops”). Polyglot programming is useful because each language has different inherent strengths. A language like Scala may be better suited for large “Big Data” processing tasks, whereas a language like Ruby may be friendlier for shell scripts. Used together, they create a synergy. The rise of so-called *microservice* architectures, in which services written in different languages interact with each other to form complex, unified systems, is a logical outcome of polyglot programming.

Similar to its language-centric cousin, polyglot persistence happens when you use the strengths of many kinds of database in the same system, as opposed to relying on a single database (often a relational database). A basic variant of this is already quite common: using a key-value store (such as Redis) that acts as a cache for queries against a relatively slower relational database (such as PostgreSQL). Relational models, as we’ve seen in previous chapters, are ill-suited for a growing host of problems, such as graph traversal. But even these new databases shine only as a few stars in the full galaxy of requirements.

Why the sudden interest in polyglot everything? Nearly a decade ago, Martin Fowler noted<sup>a</sup> that having a single central database that multiple applications could use as a source of truth was a common pattern in software design. This once-popular database integration pattern has given way to a *middleware layer* pattern, where multiple applications instead communicate to a service layer over HTTP or a lighter protocol, such as Thrift or Protocol Buffers. This frees up the middleware service itself to rely on any number of databases or, in the case of polyglot persistence, many types of database.

a. <http://martinfowler.com/bliki/DatabaseThaw.html>

## Population

The first item of business is to populate our databases with the necessary data. We take a two-phased approach here, by first populating Redis and then populating our CouchDB SOR. We’ll be using a large dataset holding information about bands, including the names of those bands, band members, which role each band member played, and more. This dataset contains a lot of information, but we’re interested only in extracting the member or artist name, the group or band name, and their roles in that band stored as a comma-separated list. For example, *John Cooper* played in the band *Skillet* as the *Lead vocalist*, *Acoustic guitar* player, and *Bassist*.

First, download the file holding the dataset:

```
$ curl -O [data-file-url]
```

Replace [data-file-url] with the URL in the footnote below.<sup>7</sup>

This file contains a lot of information, but we are interested only in extracting the member or artist name, the group or band name, and their roles in that band stored as a comma-separated list. For example, *John Cooper* played in the band *Skillet* as the *Lead vocalist*, *Acoustic guitar* player, and *Bassist*.

```
/m/0654bxy John Cooper Skillet Lead vocalist,Acoustic guitar,Bass 1996
```

Ultimately we want to structure John Cooper and the other members of Skillet into a single CouchDB document like the following, stored at the URL `http://localhost:5984/bands/Skillet`:

```
{
  "_id": "Skillet",
  "name": "Skillet"
  "artists": [
    {
      "name": "John Cooper",
      "role": [
        "Acoustic guitar",
        "Lead vocalist",
        "Bass"
      ]
    },
    ...
    {
      "name": "Korey Cooper",
      "role": [
        "backing vocals",
        "Synthesizer",
        "Guitar",
        "Keyboard instrument"
      ]
    }
  ]
}
```

This file contains well over 100,000 band members and more than 30,000 bands. That's not many, but it's a good starting point to build your own system. Note that not all roles are documented for each artist. This is an incomplete dataset, but we can deal with that later.

7. [https://raw.githubusercontent.com/coderoshi/NCNRCSBuzzSpec/master/group\\_membership.tsv](https://raw.githubusercontent.com/coderoshi/NCNRCSBuzzSpec/master/group_membership.tsv)

## Phase 1: Data Transformation

You may wonder why we bother populating Redis and don't just dive right into populating CouchDB. Acting as an intermediary, Redis adds structure to the flat TSV data so that subsequent insertion into another database is fast. Because our plan is to create a single record per band name, Redis allows us to make a single pass through our TSV file (which lists the same band for each band member—each band member is represented in a line). Adding single members directly to CouchDB for each line in the file can cause update thrashing, where two band member lines attempt to create/update the same band document at the same time, forcing the system to reinsert when one of them fails CouchDB's version check.

The catch with this strategy is that you're limited to the constraints of Redis to hold an entire dataset in RAM—though this limit could be overcome by the simple consistent-hashing cluster you saw on Day 2.

With our data file in hand, ensure you have Node.js installed as well as the Node Package Manager (npm). Once that's all done, we need to install three NPM projects: redis, csv, and hiredis (the optional Redis C-driver you learned about on Day 2 that can greatly speed up Redis interactions).

```
$ npm install hiredis redis csv-parser cradle
```

Then, check that your Redis server is running on the default port 6379, or alter each script's `createClient()` function to point to your Redis port.

You can populate Redis by running the following Node.js script in the same directory as your TSV file, which we assume is named `group_membership.tsv`. (All of the JavaScript files we'll look at are fairly verbose, so we don't show them in their entirety. All of the code can be downloaded from the Pragmatic Bookshelf website. Here we'll just stick to the meat of each file.) Download and run the following file:

```
$ node prePopulate.js
```

This script basically iterates through each line of the TSV and extracts the artist name, the band name, and the roles they play in that band. Then it adds those values to Redis (skipping any blank values).

The format of each Redis band key is `"band:Band Name"`. The script will add this artist name to the set of artist names. So, the key `"band:Beatles"` will contain the set of values `["John Lennon", "Paul McCartney", "George Harrison", "Ringo Starr"]`. The artist keys will also contain the band name and similarly contain a set of roles. `"artist:Beatles:Ringo Starr"` will contain the set `["Drums"]`.

The other code just keeps track of how many lines we've processed and outputs the results to the screen.

#### redis/prePopulate.js

```
var stream = csv({
  separator: '\t',
  newline: '\n'
});

fs.createReadStream(tsvFilename)
  .pipe(stream)
  .on('data', function(data) {
    var
      artist = data['member'],
      band = data['group'],
      roles = buildRoles(data['role']);

    if (artist === '' || band === '') {
      trackLineCount();
      return true;
    }

    redisClient.sadd('band:' + band, artist);

    if (roles.length > 0) {
      roles.forEach(function(role) {
        redisClient.sadd('artist:${band}:${artist}', role);
      });
    }

    trackLineCount();
  })
  .on('end', function(totalLines) {
    console.log(`Total lines processed: ${processedLines}`);
    redisClient.quit();
  });
```

You can test that the code has been populating Redis by launching `redis-cli` and executing `RANDOMKEY`. We should expect a key prefixed by `band:` or `artist:` ... any value but `(nil)` is good.

Now that Redis is populated, proceed immediately to the next section. You could lose data if you turn Redis off, *unless* you chose to set a higher durability than the default or initiated a `SAVE` command.

### Phase 2: SOR Insertion

CouchDB will play the role of our system of record (SOR). If any data conflicts arise between Redis, CouchDB, or Neo4j, CouchDB wins. A good SOR should contain all of the data necessary to rebuild any other data source in its domain.



## Eric says: Nonblocking Code

Before starting this book, we were only passingly familiar with writing event-driven nonblocking applications. *Nonblocking* means precisely that: rather than waiting for a long-running process to complete, the main code will continue executing. Whatever you need to do in response to a blocking event you put inside a function or code block to be executed later. This can be by spawning a separate thread or, in our case, implementing a reactor pattern event-driven approach.

In a blocking program, you can write code that queries a database, waits, and loops through the results.

```
results = database.some_query()
for value in results
  # do something with each value
end
# this is not executed until after the results are looped...
```

In an event-driven program, you would pass in the loop as a function/code block. While the database is doing its thing, the rest of the program can continue running. Only after the database returns the result does the function/code block get executed.

```
database.some_query do |results|
  for value in results
    # do something with each value
  end
end
# this continues running while the database performs its query...
```

It took us quite some time to realize the benefits here. The rest of the program can run rather than sitting idle while it waits on the database, sure, but is this common? Apparently so, because when we began coding in this style, we noticed an order-of-magnitude decrease in latency.

We try to keep the code as simple as we can, but interacting with databases in a non-blocking way is an inherently complex process. But as we learned, it's generally a very good method when dealing with databases. Nearly every popular programming language has some sort of nonblocking library. Ruby has EventMachine, Python has Twisted, Java has the NIO library, C# has Interlace, and, of course, JavaScript has Node.js.

Ensure CouchDB is running on the default port 5984, or change the `require('http').createClient(5984, 'localhost')` line in the following code to the port number you require. Redis should also still be running from the previous section. Download and run the following file:

```
$ node populateCouch.js
```

Because phase 1 was all about pulling data from a TSV and populating Redis, this phase is all about pulling data from Redis and populating CouchDB. You

don't use any special drivers for CouchDB because it's a simple REST interface and Node.js has a simple built-in HTTP library.

In the following block of code, we perform a Redis KEYS `bands:*` to get a list of all band names in our system. If we had a *really* big dataset, we could add more scoping (for example, `bands:A*` to get only band names starting with *a*, and so on). Then, for each of those bands we fetch the set of artists and extract the band name from the key by removing the prefix *bands:* from the key string.

```
redis/populateCouch.js
redisClient.keys('band:*', function(err, bandKeys) {
  totalBands = bandKeys.length;
  var
    readBands = 0,
    bandsBatch = [];

  bandKeys.forEach(function(bandKey) {
    // substring of 'band:'.length gives us the band name
    var bandName = bandKey.substring(5);
    redisClient.smembers(bandKey, function(err, artists) {
```

Next, we get all of the roles for every artist in this band, which Redis returns as an array of arrays (each artists role is its own array). You can do this by batching up Redis SMEMBERS commands into an array called `roleBatch` and executing them in a single MULTI batch. Effectively, that would be executing a single pipelined request like this:

```
MULTI
  SMEMBERS "artist:Beatles:John Lennon"
  SMEMBERS "artist:Beatles:Ringo Starr"
EXEC
```

From there, a batch of 50 CouchDB documents is made. We build a batch of 50 because we then send the entire set to CouchDB's `/_bulk_docs` command, allowing us very, very fast insertion.

```
redis/populateCouch.js
redisClient.
  multi(roleBatch).
  exec(function(err, roles) {
    var
      i = 0,
      artistDocs = [];

    // build the artists sub-documents
    artists.forEach(function(artistName) {
      artistDocs.push({ name: artistName, role : roles[i++] });
    });
```

```
// add this new band document to the batch to be executed later
bandsBatch.push({
  _id: couchKeyify(bandName),
  name: bandName,
  artists: artistDocs
});
```

With the population of the bands database, we now have in a single location all of the data our system requires. We know the names of many bands, the artists who performed in them, and the roles they played in those bands.

Now would be a good time to take a break and play around with our newly populated bands system of record in CouchDB at [http://localhost:5984/\\_utils/database.html?bands](http://localhost:5984/_utils/database.html?bands).

## Relationship Store

Next on the docket is our Neo4j service that we'll use to track relationships between artists and the roles they play. We could certainly query CouchDB outright by creating views, but we are rather limited on complex queries based on relationships. If Wayne Coyne from the Flaming Lips loses his theremin before a show, he could ask Charlie Clouser from Nine Inch Nails, who also plays a theremin. Or we could discover artists who have many overlapping talents, even if they performed different roles in different bands—all with a simple node walk.

With our initial data in place, now we need to keep Neo4j in sync with CouchDB should any data ever change on our system of record. So, we'll kill two birds by crafting a service that populates Neo4j with any changes to CouchDB since the database was created.

We also want to populate Redis with keys for our bands, artists, and roles so we can quickly access this data later. Happily, this includes all data that we've already populated in CouchDB, thus saving us a separate initial Neo4j and Redis population step.

Ensure that Neo4j is running on port 7474, or change the appropriate `createClient()` function to use your correct port. CouchDB and Redis should still be running. Download and run the following file. This file will continue running until you shut it down.

```
$ node graphSync.js
```

This server just uses the continuous polling example we saw in the CouchDB chapter to track all CouchDB changes. Whenever a change is detected, we do two things: populate Redis and populate Neo4j. This code populates Redis

by cascading callback functions. First it populates the band as "band-name:Band Name". It follows this pattern for artist name and roles.

This way, we can search with partial strings. For example, `KEYS band-name:Bea*` could return this: Beach Boys, Beastie Boys, Beatles, and so on.

`redis/graphSync.js`

```
function feedBandToRedis(band) {
  redisClient.set(`band-name:${band.name}`, 1);
  band.artists.forEach(function(artist) {
    redisClient.set(`artist-name:${artist.name}`, 1);
    artist.role.forEach(function(role){
      redisClient.set(`role-name:${role}`, 1);
    });
  });
}
```

The next block is how we populate Neo4j. We created a driver that you can download as part of this book's code, named `neo4jCachingClient.js`. It just uses Node.js's HTTP library to connect to the Neo4j REST interface with a bit of rate-limiting built in so the client doesn't open too many connections at once. Our driver also uses Redis to keep track of changes made to the Neo4j graph without having to initiate a separate query. This is our third separate use for Redis—the first being as a data transformation step for populating CouchDB, and the second we just saw earlier, to quickly search for band values.

This code creates band nodes (if they need to be created), then artist nodes (if they need to be created), and then roles. Each step along the way creates a new relationship, so The Beatles node will relate to John, Paul, George, and Ringo nodes, who in turn each relate to the roles they play.

`redis/graphSync.js`

```
function feedBandToNeo4j(band, progress) {
  var
    lookup = neo4jClient.lookupOrCreateNode,
    relate = neo4jClient.createRelationship;

  lookup('bands', 'name', band.name, 'Band', function(bandNode) {
    progress.emit('progress', 'band');
    band.artists.forEach(function(artist) {
      lookup('artists', 'name', artist.name, 'Artist', function(artistNode) {
        progress.emit('progress', 'artist');
        relate(bandNode.metadata.id, artistNode.self, 'member', function() {
          progress.emit('progress', 'member');
        });
      });
      artist.role.forEach(function(role){
        lookup('roles', 'role', role, 'Role', function(roleNode) {
          progress.emit('progress', 'role');
          relate(artistNode.metadata.id, roleNode.self, 'plays', function() {
            progress.emit('progress', 'plays');
          });
        });
      });
    });
  });
}
```



Let this service keep running in its own window. Every update to CouchDB that adds a new artist or role to an existing artist will trigger a new relationship in Neo4j and potentially new keys in Redis. As long as this service runs, they should be in sync.

Open your CouchDB web console and open a band. Make any data change you want to the database: add a new band member (make yourself a member of the Beatles!), or add a new role to an artist. Keep an eye on the graphSync output. Then fire up the Neo4j console and try finding any new connections in the graph. If you added a new band member, they should now have a relationship with the band node or new role if that was altered. The current implementation does not remove relationships—though it would not be a complete modification to add a Neo4j DELETE operation to the script.

## The Service

This is the part we've been building up to. We're going to create a simple web application that allows users to search for a band. Any band in the system will list all of the band members as links, and any clicked band member link will list some information about the artist—namely, the roles they play. In addition, each role the artist plays will list every other artist in the system who also plays that role.

For example, searching for Led Zeppelin would give you Jimmy Page, John Paul Jones, John Bonham, and Robert Plant. Clicking Jimmy Page will list that he plays guitar and also many other artists who play guitar, like The Edge from U2.

To simplify our web app creation a bit, we'll need two more node packages: bricks (a simple web framework) and mustache (a templating library).

```
$ npm install bricks mustache neo4j-driver
```

As in the previous sections, ensure you have all of the databases running, and then start up the server. Download and run the following code:

```
$ node band.js
```

The server is set to run on port 8080, so if you point your browser to <http://localhost:8080/>, you should see a simple search form.

Let's take a look at the code that will build a web page that lists band information. Each URL performs a separate function in our little HTTP server. The first is at <http://localhost:8080/band> and accepts any band name as a parameter.

```

redis/bands.js
appServer.addRoute("/band$", function(req, res) {
  var
    bandName = req.param('name'),
    bandNodePath = couchUtil.couchKeyify(bandName),
    membersCypherQuery =
      `MATCH (Band {name: "${bandName}"})-[:member*1..3]-(b:Band) ` +
      `RETURN DISTINCT b LIMIT 10`;

  getCouchDoc(bandNodePath, res, function(couchDoc) {
    var artists = couchDoc && couchDoc['artists'];

    cypher(membersCypherQuery, function(bandsGraphData) {
      var bands = [];
      bandsGraphData.data.forEach(function(band) {
        bands.push(band[0].data.name);
      });

      var values = { band: bandName, artists: artists, bands: bands };

      var template = `
        <h2>{{band}} Band Members</h2>
        <ul>
          {{#artists}}
            <li><a href="/artist?name={{name}}">{{name}}</a></li>
          {{/artists}}
        </ul>
        <h3>You may also like</h3>
        <ul>
          {{#bands}}
            <li><a href="/band?name={{.}}">{{.}}</a></li>
          {{/bands}}
        </ul>
      `;

      writeTemplate(res, template, values);
    });
  });
});

```

If you enter the band Nirvana in the search form, your URL request will be `http://localhost:8080/band?name=Nirvana`. This function will render an HTML page (the overall template is in an external file named `template.html`). This web page lists all artists in a band, which it pulls directly from CouchDB. It also lists some suggested bands, which it retrieves from a Gremlin query against the Neo4j graph. The Gremlin query is like this for Nirvana:

```
g.V.filter{it.name=="Nirvana"}.out("member").in("member").dedup.name
```

Or in other words, from the Nirvana node, get all unique names whose members are connected to Nirvana members. For example, Dave Grohl played in Nirvana and the Foo Fighters, so Foo Fighters will be returned in this list.

The next action is the `http://localhost:8080/artist` URL. This page will output information about an artist.

`redis/bands.js`

```
appServer.addRoute("/artist$", function(req, res) {
  var
    artistName = req.param('name'),
    rolesCypherQuery = `MATCH (Artist {name: "${artistName}"})` +
      `-[[:plays]]-(r:Role) RETURN r`,
    bandsCypherQuery = `MATCH (Artist {name: "${artistName}"})` +
      `-[[:member]]-(b:Band) RETURN b`;

    cypher(rolesCypherQuery, function(rolesGraphData) {
      cypher(bandsCypherQuery, function(bandsGraphData) {
        var
          roles = [],
          bands = [];

        rolesGraphData.data.forEach(function(role) {
          roles.push(role[0].data.role);
        });

        bandsGraphData.data.forEach(function(band) {
          bands.push(band[0].data.name);
        });

        var values = { artist: artistName, roles: roles, bands: bands };

        var template = `
          <h3>{{artist}} Performs these Roles</h3>
          <ul>
            {{#roles}}
              <li>{{.}}</li>
            {{/roles}}
          </ul>
          <h3>Play in Bands</h3>
          <ul>
            {{#bands}}
              <li><a href="/band?name={{.}}">{{.}}</a></li>
            {{/bands}}
          </ul>
        `;
        writeTemplate(res, template, values);
      });
    });
});
```

Two Gremlin queries are executed here. This first outputs all roles a member plays, and the second is a list of bands that person played in. For example, Jeff Ward (`http://localhost:8080/artist?name=Jeff%20Ward`) would be listed as playing the role Drummer and in the bands Nine Inch Nails and Ministry.

A cool feature of the previous two pages is that we render links between these values. The artist list in the /bands page links to the chosen /artist page, and vice versa. But we could make searching a bit easier.

**redis/bands.js**

```
appServer.addRoute("/search$", function(req, res) {
  var query = req.param('term');

  redisClient.keys(`band-name:${query}*`, function(error, keys) {
    var bands = [];
    keys.forEach(function(key) {
      bands.push(key.replace("band-name:", ' '));
    });
    res.write(JSON.stringify(bands));
    res.end();
  });
});
```

Here we just pull all keys from Redis that match the first part of the string, such as "Bea\*" as described previously. The function then outputs the data as JSON. The template.html file links to the jQuery code necessary to make this function as an autocomplete feature on the rendered search box.

## Expanding the Service

This is a fairly little script for all of the bare-bones work we're doing here. You may find many places you want to extend. Notice that the band suggestion is only first-order bands (bands the current band's members have performed in); you can get interesting results by writing a query to traverse second-order bands, like this: `g.V.filter{it.name=='Nine Inch Nails'}.out('member').in('member').dedup.loop(3){ it.loops <= 2 }.name`.

You may also note that we do not have a form where someone can update band information. Adding this functionality could be fairly simple because we already wrote CouchDB population code in the `populateCouch.js` script, and populating CouchDB will automatically keep Neo4j and Redis eventually consistent as long as the `graph_sync.js` service is running.

If you enjoy playing with this kind of polyglot persistence, you could take this even further. You could add a PostgreSQL data warehouse<sup>8</sup> to transform this data into a star schema—allowing for different dimensions of analysis, such as most commonly played instrument or average numbers of total members in a band vs. total instruments. You could add a CouchDB server to store information about the music associated with each band, an HBase server to build a messaging system that enables users to keep track of their historical likes/dislikes, or a MongoDB extension to add a geographic element to this service.

8. [http://en.wikipedia.org/wiki/Data\\_warehouse](http://en.wikipedia.org/wiki/Data_warehouse)

Or, redesign this project entirely with a different language, web framework, or dataset. There are as many opportunities to extend this project as there are combinations of databases and technologies to create it—a Cartesian product of all open source.

### Day 3 Wrap-Up

Today was a big day—so big, in fact, we wouldn't be surprised if it took several days to complete. But this is a little taste of the future of data management systems, as the world strolls away from the *one large relational database* model to a *several specialized databases* model. We also glued these databases together with some nonblocking code, which, though not a focus of this book, also seems to be where database interaction is headed in the development space.

The importance of Redis in this model should not be missed. Redis certainly doesn't provide any functionality these databases don't supply individually, but it does supply speedy data structures. We were able to organize a flat file into a series of meaningful data structures, which is an integral part of both data population and transportation. And it did this in a fast and simple-to-use way.

Even if you're not sold on the whole polyglot persistence model, you should certainly consider Redis for any system.

### Day 3 Homework

#### Do

1. Alter the importer steps to also track a band member's start and end dates with the band. Track that data in the artist's CouchDB subdocument. Display this information on the artist's page.
2. Add MongoDB into the mix by storing a few music samples into GridFS, whereby users can hear a song or two related to a band. If any song exists for a band, add a link to the web app.

### Wrap-Up

The Redis key-value (or data structure) store is light and compact, with a variety of uses. It's akin to one of those multitools composed of a knife, can opener, and other bits and bobs like a corkscrew—Redis is good to have around for solving a variety of odd tasks. Above all, Redis is fast, simple, and as durable as you choose. While rarely a standalone database, Redis is a perfect complement to any polyglot ecosystem as an ever-present helper for transforming data, caching requests, or managing messages by way of its blocking commands.

## Redis's Strengths

The obvious strength of Redis is speed, like so many key-value stores of its ilk. But more than most key-value stores, Redis provides the ability to store complex values such as lists, hashes, and sets, and retrieve them based on operations specific to those datatypes. Beyond even a data structure store, however, Redis's durability options allow you to trade speed for data safety up to a fairly fine point. Built-in master-slave replication is another nice way of ensuring better durability without requiring the slowness of syncing an append-only file to disk on every operation. Additionally, replication is great for very high-read systems.

## Redis's Weaknesses

Redis is fast largely because it resides in memory. Some may consider this cheating because, of course, a database that never hits the disk will be fast. A main memory database has an inherent durability problem; namely, if you shut down the database before a snapshot occurs, you can lose data. Even if you set the append-only file to disk sync on every operation, you run a risk with playing back expiry values because time-based events can never be counted on to replay in exactly the same manner—though, in fairness, this case is more hypothetical than practical.

Redis also does not support datasets larger than your available RAM (Redis is removing virtual memory support), so its size has a practical limitation. Although there is a Redis Cluster currently in development to grow beyond a single machine's RAM requirements, anyone wanting to cluster Redis must currently roll their own with a client that supports it (like the Ruby driver we used in Day 2).

## Parting Thoughts

Redis is chock-full of commands—more than 120 of them. Most commands are straightforward enough to understand by their names alone, once you get used to the idea that seemingly random letters will be removed (for example, INCRBY) or that mathematical precision can sometimes be more confusing than helpful (for example, ZCOUNT, or sorted set count, vs. SCARD, or set cardinality).

Redis is already becoming an integral part of many systems. Several open source projects rely on Redis, from Resque, a Ruby-based asynchronous job queueing service, to session management in the Node.js project SocketStream. Regardless of the database you choose as your SOR, you should certainly add Redis to the mix.